Xiaomi CYBERWAM: Multi-Stream World Action Model for the BEHAVIOR-1K 2026 Challenge

Hao Li, Hao Liu, Zhengjun Wang, Diyun Xiang, Hangjun Ye, Tao Kong, Zhiying Du

Xiaomi Robotics

Project page: `https://xiaomi-research.github.io/cyberwam/`

Abstract

World action models (WAMs) predict future observations together with actions on top of a pretrained video diffusion backbone. They inherit the temporal and physical priors of large-scale video pretraining and receive dense visual supervision beyond sparse action labels, which makes them an appealing basis for long-horizon manipulation. We build our solution to the BEHAVIOR-1K 2026 challenge, 100 long-horizon household tasks for a bimanual mobile manipulator, on this idea. Our main model is a multi-stream WAM based on Wan2.2-TI2V-5B that jointly denoises RGB video, DINO features, point maps and robot actions, conditioned on three camera views, proprioception and a task prompt. Wan2.2 has been pretrained on large amounts of video and already captures much of the temporal and physical structure of the world, yet building a good WAM on top of it is not straightforward: the training and inference settings, and the values chosen for them, have a marked effect on the final score. Through extensive experiments we arrive at a training recipe and an inference procedure that together raise the score from 0.246 for the baseline to 0.606 Q on the full 1000-rollout evaluation, close to a 2.5-fold improvement. On the 50 tasks of the 2025 edition, which are also part of the 2026 edition, the model reaches 0.587 Q, more than twice the 0.26 Q of the 2025 winner, a $\pi_{0.5}$-based solution; the two editions differ in the amount of training data, so this comparison is indicative.

1 Introduction

Household robots have to carry out tasks that last for many minutes, in scenes the size of a house, using two arms, a mobile base and a torso. The BEHAVIOR-1K 2026 challenge turns this into a benchmark: 100 full-length household tasks in a physically simulated environment, solved from RGB, depth and proprioception, with 20 000 teleoperated demonstrations (about 1 950 hours) for training [1]. The tasks are long: some of the rollouts we evaluated run for more than twenty minutes of simulated time. They are also hard in ways that short pick-and-place benchmarks are not. Errors accumulate over thousands of control steps, many task states look alike before and after a step is done, the demonstrations contain few recoveries, and the same step can be carried out in several equally valid ways [2]. The score gives partial credit for the goal conditions that are satisfied. In the 2025 edition, which had 50 tasks, the winning solution reached a q-score of 0.26 on the public and private leaderboards [2], and the runner-up reports 0.345 on a validation set [3].

Both of those solutions fine-tune a vision-language-action (VLA) model. We take a different route and build on a world action model (WAM), which predicts the future of the scene together with the actions. A WAM is built on a pretrained video diffusion model, so it inherits temporal and physical priors from large-scale video, and it is trained to denoise future observations as well as actions, which gives it a dense learning signal in addition to sparse action labels [4]. Our model starts from Fast-WAM [4] and the Wan2.2-TI2V-5B video backbone [5]. Following Flex-π [6], it predicts not only future RGB video but also point maps computed from depth and DINOv3 features [7], and it denoises all of them together with a chunk of 23-dimensional actions that is conditioned on three camera views, the proprioceptive state and a task prompt (Figure 1).

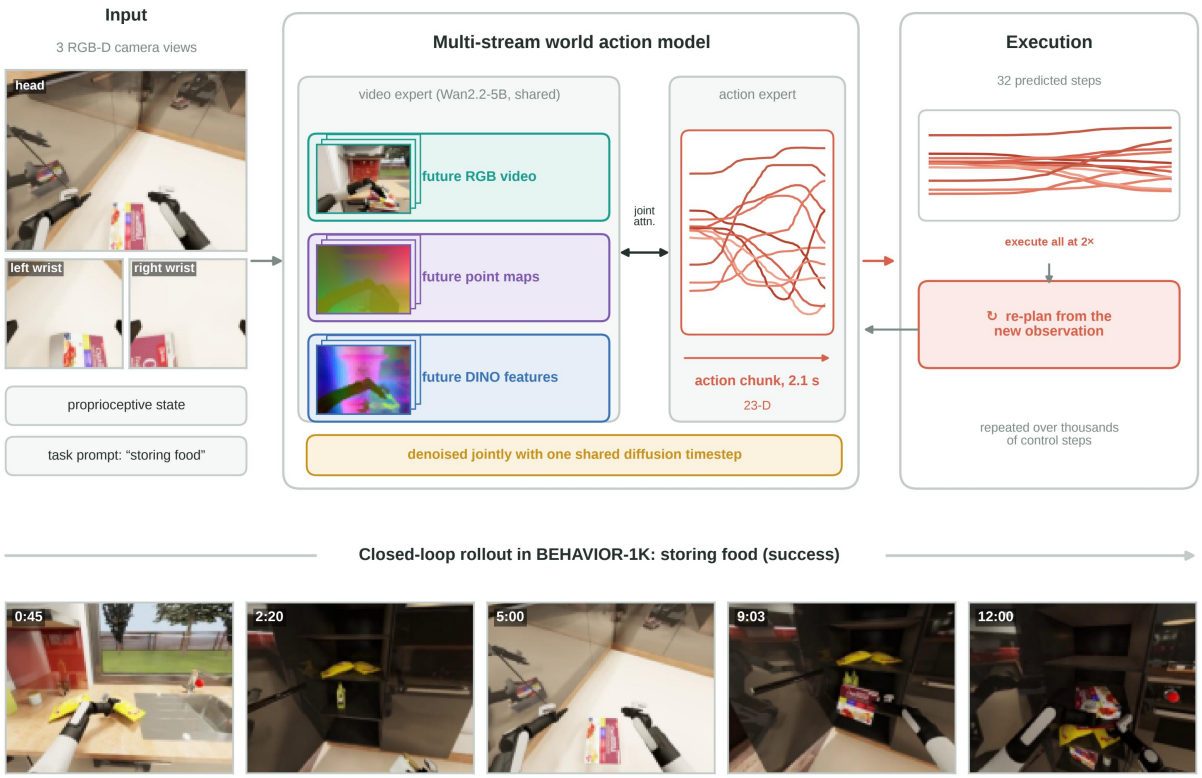


Figure 1: Overview. From three RGB-D camera views, the proprioceptive state and a task prompt, our multi-stream world action model denoises future RGB video, point maps and DINO features together with a 2.1 s action chunk, using one diffusion timestep for all streams. The robot executes the whole chunk at twice the demonstrated speed, in about 1 s, and then re-plans. The stream thumbnails and the action curves are taken from training data. *Bottom:* keyframes of a successful rollout of our model on *storing food*, a 12-minute task in which the robot moves several food items into a cabinet.

Making this work on BEHAVIOR-1K took a combination of model design, training and execution choices, each checked on the official metric. On the model side, the RGB, point-map and DINO streams share one video expert and each add only their own input projection and output head, with the point-map ones initialized from the RGB ones, so the extra streams add little to the parameter count. An attention mask lets the action tokens read the predicted future but keeps the video tokens from seeing the actions, so the predicted futures do not depend on the action being denoised (Section 3). On the training side, we found that giving each stream its own diffusion timestep leaves almost all training samples far from the diagonal that sampling follows, and that the fraction shrinks from 19% with two streams to 0.37% with four. We therefore use a single timestep for all streams, and we also drop the proprioceptive state on a fraction of the samples so that the policy relies less on it. On the execution side, we play the predicted actions back at twice the demonstrated speed and re-plan after executing each whole chunk. Starting from a baseline that scores 0.246 Q on all 1000 public test rollouts, the multi-stream model, the faster execution and the training settings together raise the score to 0.606 Q (Section 4.2). On the 50 tasks that the 2026 edition shares with the 2025 edition, the final model reaches 0.587 Q, more than twice the 0.26 Q of the 2025 winner (Section 4.2).

2 Related Work

From video prediction to world action models. Early work used video generation as a planner: UniPi casts a policy as text-conditioned video generation and recovers the actions from the generated frames [8]. Later work moved from generating pixels to using what the video

model has learned. The Video Prediction Policy conditions an action head on the predictive representations inside a video diffusion model, which it fine-tunes on robot and human manipulation data [9], and the Unified Video Action model learns a joint video–action latent that can be decoded without generating video at inference [10]. World action models (WAMs) go one step further and denoise future observations together with actions in a single model. Genie Envisioner builds a video-generative platform with a flow-matching action decoder [11], Cosmos Policy adapts a pretrained video model into a policy by post-training it on robot demonstrations without architectural changes [12], and Motus and LingBot-VA learn frame prediction and control together with Mixture-of-Transformers backbones [13, 14]. Fast-WAM, on which our model is built, asks whether the future has to be generated at test time at all. It keeps video co-training but skips future prediction at inference, and finds that removing the co-training hurts much more than removing the generation [4]. Our work follows this line, using a Wan2.2 video backbone [5] with a separate action expert.

Predicting more than appearance. Most WAMs predict future RGB only, so their prediction target is pixel appearance. Several recent works add geometry or semantics. SA-WAM predicts depth together with RGB and actions and reuses the frozen video tokenizer for it [15]; PointAction predicts RGB and dynamic 3D point maps and decodes actions from them [16]; RynnWorld-4D produces RGB, depth and optical flow in one diffusion process [17]; and other works add geometric priors to bimanual or world-action policies [18–20]. Flex-π is the closest to our model: it encodes RGB and point maps with the same frozen video VAE, adds DINOv3 semantic features [7], denoises all streams jointly with actions in a Mixture-of-Transformers, and uses per-stream dropout so that one checkpoint can run on a subset of streams [6]. We adopt this multi-stream design, with a fixed set of streams, and ask how it behaves on long-horizon household tasks. Among the works above, those whose evaluations we checked report tabletop simulation benchmarks and real-robot tasks, such as LIBERO, RoboTwin, RoboCasa and a UR5 arm [4, 15], and our search did not find one that reports results on BEHAVIOR-1K.

Vision-language-action policies and action generation. The main alternative to WAMs is the vision-language-action (VLA) model. The π_0 family attaches an action expert trained with flow matching [21] to a pretrained vision-language model [22, 23], in the line of diffusion policies [24], and, like action chunking with transformers [25], they predict a chunk of future actions at once. The Mixture-of-Transformers design that WAMs and our model share gives each modality its own weights and lets them interact through global self-attention [26]. A chunked policy has two further choices at execution time: how much of the chunk to execute before re-planning, and how fast to play it back. We measure the effect of the playback speed (Section 4.2).

BEHAVIOR-1K and its challenges. BEHAVIOR-1K defines 1 000 everyday household activities in a physically simulated environment [1]. Its 2025 challenge asked for 50 full-length tasks; the winning solution fine-tuned $\pi_{0.5}$ with correlated noise for flow matching, among other changes, and reached a q-score of 26% [2], and the runner-up, also built on $\pi_{0.5}$, studied training techniques and data [3]. The 2026 challenge doubles the number of tasks to 100 and provides 20 000 teleoperated demonstrations; its reference baselines are $\pi_{0.5}$ and GR00T N1.7. Both 2025 reports start from a VLA and improve its training and inference. The winner also speeds up execution, compressing 26 predicted actions into 20 steps with cubic splines (a 1.3× speed-up), and reports that small changes to inference parameters made little difference while extreme ones, such as a 2× speed-up, lowered the score [2]. We instead start from a WAM, and we find that a 2× playback helps our model (Section 4.2), so the best speed-up may depend on the policy and on how the actions are resampled.

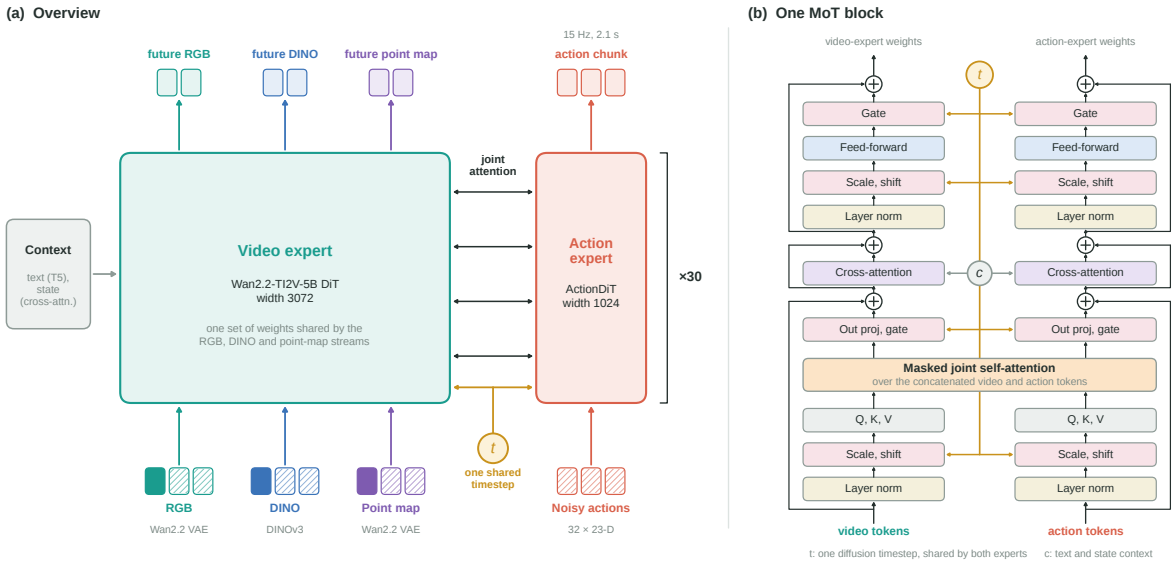


Figure 2: The multi-stream world action model (CYBERWAM). **(a)** RGB, DINO and point-map tokens (a clean anchor plus noised future latents) enter a video expert whose weights are shared by the three visual streams; noised action tokens enter a narrower action expert (widths drawn to scale, 3072 vs. 1024). The two experts attend jointly in every block, cross-attend to text and state context, and are conditioned on one diffusion timestep t that is shared by all streams. **(b)** One of the 30 blocks: each expert keeps its own normalization, projection, cross-attention and feed-forward weights, while self-attention runs over the concatenated tokens of both experts under the mask of Figure 4.

3 Method

This section describes the final system. How each design was arrived at, and what it was worth in score, is deferred to Section 4; data processing, optimization and deployment details are given in Appendix A.

3.1 Overview and Problem Formulation

Task interface. The robot is a bimanual mobile manipulator with a torso. At each decision step the policy receives RGB and depth from a head camera (720×720) and two wrist cameras (480×480 each), a 23-D proprioceptive state s and a task prompt ℓ, and returns a chunk of 23-D actions: base velocity (3, a normalized planar twist), and absolute joint-position targets for the torso (4), left arm (7) and right arm (7), plus one command per gripper (left and right, 1 each). Gripper commands are binary (± 1, open or closed). The 23-D state has the same layout: measured base velocity, torso and arm joint positions, and the opening width of each gripper. The policy runs at 15 Hz while the simulator is controlled at 30 Hz, and a chunk covers 32 predicted steps (about 2.1 s).

World action model. We cast control as conditional generation of the near future. Given the current observation o, the state s and the prompt ℓ, the model generates jointly an action chunk $a \in \mathbb{R}^{32\times 23}$ and the future observations that should follow from it, represented in three visual streams: RGB video latents z^{rgb}, DINO features z^{dino} and point-map latents z^{pm}. Only the actions are executed; the predicted futures act as a dense training signal that ties the actions to their visual consequences.

Flow matching. Every stream $x \in \{z^{\text{rgb}}, z^{\text{dino}}, z^{\text{pm}}, a\}$ is generated with rectified flow. A clean sample x_0 and Gaussian noise ϵ are interpolated at noise level $\sigma \in [0, 1]$ as $x_\sigma = (1 -$

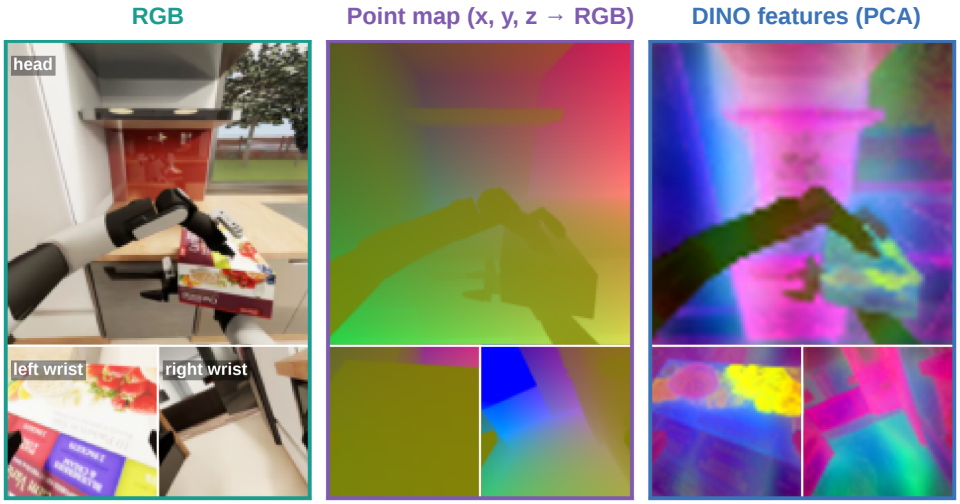


Figure 3: The three visual streams of one training frame (*storing food*). **Left:** the RGB image, with the head view on top and the two wrist views below. **Middle:** the point map, with the camera-frame coordinates (x, y, z) shown as colour channels. **Right:** the first three principal components of the DINOv3 patch features. For legibility the streams are shown at a higher resolution than the model uses: the views at twice the size of the 384×256 image, and DINOv3 at 56×56 patches per view, whereas the model predicts DINO features on a 6×6 grid per view.

$\sigma)\, x_0 + \sigma\, \epsilon$, and the network v_θ regresses the velocity $\epsilon - x_0$. Noise levels are drawn as

$$\sigma = \frac{\lambda u}{1 + (\lambda - 1)\, u}, \qquad u \sim \mathcal{U}(0, 1), \tag{1}$$

where the shift λ moves mass towards high noise; at inference the same map is applied to an evenly spaced grid of u. The anchor (current) frame of each visual stream is kept clean and serves as the condition. The training loss is the sum of the four per-stream mean-squared errors with equal weights, each weighted by a timestep-dependent factor; padded action steps and padded frames are excluded. The DINO branch predicts the clean features $\hat{x}_0$, which are converted to a velocity $(x_\sigma - \hat{x}_0)/\sigma$ before the loss.

3.2 Multi-stream world action model

Our main model, CYBERWAM, builds on the Wan2.2-TI2V-5B video diffusion transformer and adds a second, narrower action expert in a mixture-of-transformers (MoT) design: each expert has its own weights, and the experts attend to each other through joint attention in every block (Figure 2). It denoises the four streams together. The choice of streams and their joint denoising with the actions follow Flex-π [6]; we use a fixed set of streams.

Visual streams. The three camera views are tiled into one 384×256 image (head view on top, the two wrist views below) and encoded by the frozen, pretrained Wan2.2 VAE into RGB latents. Depth is converted to a point map, tiled the same way and encoded by the same VAE. A frozen DINOv3 ViT-B/16 encodes the RGB frames into features on a 6×6 grid per view. The RGB, DINO and point-map streams all run through the *same* video-expert blocks (30 layers, width 3072); each stream has only its own input projection and output head, and those of the point-map stream are initialized from the RGB ones. The model sees one anchor frame and predicts two future latents, which correspond to nine video frames sampled at 3.75 Hz and compressed 4× in time by the VAE (Figure 3).

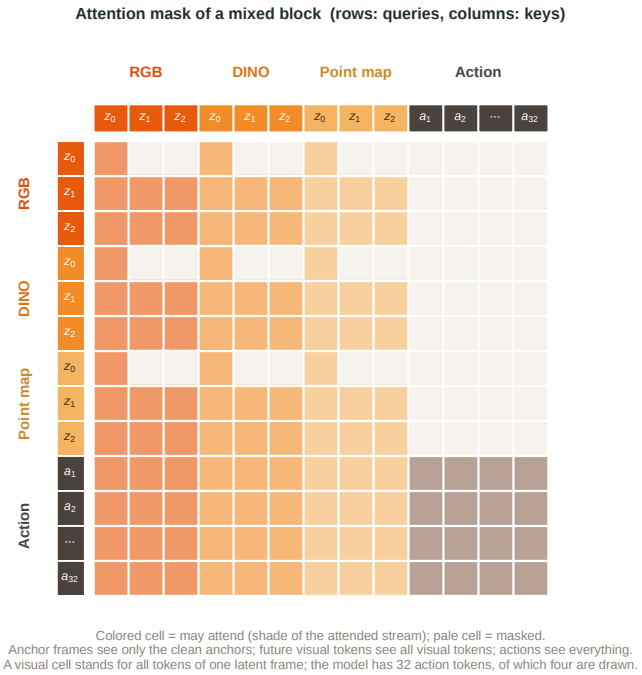


Figure 4: Attention mask of a mixed block, with one anchor (z_0) and two future latent frames (z_1, z_2) per visual stream. Each visual cell stands for all tokens of one latent frame, and the four action cells stand for the 32 action tokens of the model ($a_1, \ldots, a_{32}$). Anchor tokens attend only to the clean anchors of the visual streams, future visual tokens attend to all visual tokens, and action tokens attend to everything.

Action stream and conditioning. Actions are denoised by a separate action expert (ActionDiT, 30 layers, width 1024), which is initialized from the Wan2.2 DiT by linear interpolation. It produces 32 action steps at 15 Hz. The task prompt is encoded by T5 (precomputed and cached, 128 tokens) and the proprioceptive state is encoded into one extra token; both are appended to the context that the experts cross-attend to.

Routing across streams. The multi-stream design allows each visual stream to be present as input and requested as output. In our final configuration all visual streams are always present and always predicted, so routing is fixed. We use the HBridge layer schedule: attention is stream-local in the bottom and top quarters of the layers and mixed across streams in the middle half (Figure 4).

3.3 Shared Diffusion Timestep across Streams

Let $u_k \in [0, 1]$ be the position of stream k on the noise schedule of Equation (1) (0 clean, 1 pure noise). With one visual and one action stream, drawing an independent timestep per stream spreads the training samples over the unit square, but at inference both streams start from noise and advance on the same step index, so sampling only walks the diagonal $u_{\text{vis}} = u_{\text{act}}$ (Figure 5a). Independent sampling makes the action expert robust to visual features at any noise level, but spends most of the training on noise combinations that inference never visits, and the diagonal is learned slowly. The effect grows with the number of streams. Our model has four, so independent timesteps fill the hypercube $[0, 1]^4$ while inference still follows its one-dimensional diagonal: only 0.37% of such samples have all four timesteps within 0.1 of each other, against 19% with two streams (Figure 5c).

We therefore train on the diagonal. Each sample draws a single u, and the resulting σ is applied to all four streams (Figure 5b). The streams must also share the schedule: by default

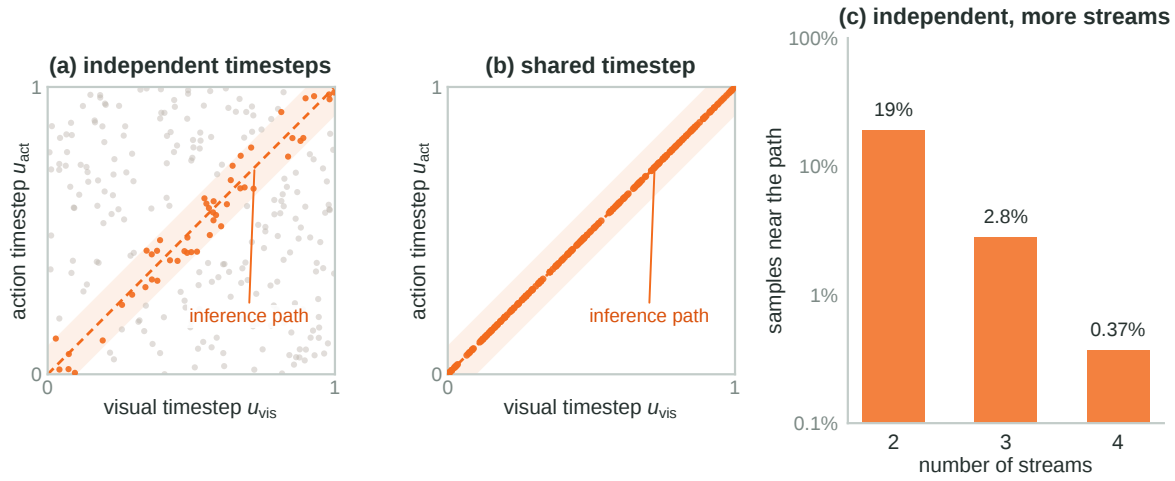


Figure 5: Training timesteps versus the inference path. **(a)** Independent timesteps cover the whole square; only the orange samples lie near the diagonal that sampling follows. **(b)** A shared timestep places every sample on that diagonal. **(c)** With independent timesteps, the fraction of samples whose timesteps all lie within 0.1 of each other drops from 19% for two streams to 0.37% for the four streams of our model.

the action stream uses shift $\lambda = 1$ and the visual streams $\lambda = 5$, so even a shared step index gives them different noise levels. We use $\lambda = 5$ for actions in training and inference. The model gives up robustness to unequal noise levels, which inference does not need, and adds no parameters or compute.

3.4 Proprioceptive State Dropout

Motivation. The proprioceptive state can act as a shortcut. Within a demonstration, consecutive joint positions are highly correlated, so a policy can lower its training loss by extrapolating the state instead of reading the scene. We expect such a policy to follow the motion it is already in and reacts poorly when the situation calls for a change, for example after a failed grasp or when the closed-loop state drifts from the states seen in the demonstrations.

Dropout on the state token. During training we replace the normalized state by zero on a random 10% of the samples. Since the state is z-scored, zero is the training-set mean, a neutral value rather than an out-of-range one. The state token and its attention slot are kept, so the context length and the attention pattern do not change; only the content of the token is removed. On these samples the model must infer the robot configuration from the images and the predicted futures, which pushes the action expert to use visual evidence on the remaining samples as well. At inference the true state is always provided.

4 Experiments

We first describe the evaluation protocol and the models (Section 4.1). Section 4.2 gives the main result as one line from the baseline to our final model. Section 4.3 tests the final model on a scene layout that never occurs in the training data.

4.1 Experimental setup

Metric. We report the official BEHAVIOR-1K score Q. An episode that completes the task scores 1; otherwise it scores the fraction of goal conditions that were false at the start and

are true at the end (taking the best of the task's alternative goal definitions). Q is averaged over the instances of each task and then over tasks. An episode counts as a success only when $Q = 1$. The official evaluation stops an episode after 1.5 times the average length of the human demonstrations of its task.

Evaluation set. The challenge provides 10 public test instances for each of the 100 tasks. All results in this section are evaluated on all of them, i.e. 1000 rollouts per model, which we call Full-1000.

Models. We use two model configurations.

- **Fast-WAM**: the original single-stream design [4]. The video expert (Wan2.2-TI2V-5B, width 3072) and the action expert (width 1024) form the same MoT as in Section 3.2, but the only visual stream is RGB video. It is our baseline.
- **CYBERWAM**: the multi-stream model of Section 3, which adds the point-map and DINO streams to the video expert. It predicts 2.1 s (two future latents and a 32-step action chunk). Training and inference details are given in Appendix A.

4.2 Main result

Table 1 and Figure 6 go from the baseline to our final model.

1. **Baseline.** The single-stream model in its unconditional mode, in which the action tokens do not attend to the predicted future, with a 2.1 s prediction horizon. It is executed at the demonstrated speed (1×) with execution horizon 16, i.e. the policy executes the first 16 of the 32 predicted actions and then re-plans. It scores 0.246 Q.
2. **Faster execution.** The same checkpoint is executed at twice the demonstrated speed (`speed_2x`); only the playback speed changes. Under the time limit of 1.5 times the average demonstration length, a policy that moves at the demonstrated speed has little time left for retries, and the teleoperated demonstrations are often slower than the motion requires. The policy predicts actions at 15 Hz and the simulator runs at 30 Hz, so `speed_2x` applies each action for one control step instead of two. Q rises from 0.246 to 0.279 and successes from 77 to 124.
3. **Multi-stream model.** We replace the baseline by the multi-stream model, trained without the settings of step 5, and keep the deployment of the baseline (1×, execution horizon 16). Q rises from 0.246 to 0.331.
4. **Faster execution on the multi-stream model.** The same checkpoint is executed with `speed_2x` and runs its whole 32-step chunk before re-planning (execution horizon 32). Only the deployment changes, and Q rises from 0.331 to 0.456.
5. **Training settings.** We retrain the multi-stream model with the same architecture and data, but with a shared diffusion timestep (Section 3.3) and state dropout (Section 3.4). Q rises from 0.456 to 0.606 and successes from 258 to 412.

Overall, the score rises from 0.246 to 0.606 Q, close to 2.5 times the baseline. The multi-stream model adds 0.085 over the baseline at the same deployment, faster execution adds 0.033 on the baseline and 0.125 on the multi-stream model, and the training settings add 0.150.

Comparison with the 2025 winner. The 2025 edition had 50 tasks and 10 000 demonstrations, and its winning solution, which fine-tunes $\pi_{0.5}$, reached 0.26 Q [2]. These 50 tasks are also part of the 2026 edition (task indices 0–49 in Appendix C). Averaged over them, our final model reaches 0.587 Q, with 186 successful episodes out of 500, more than twice the score of the 2025 winner; on the other 50 tasks it reaches 0.625 Q. The two numbers come from different editions, with different amounts of training data (10 000 and 20 000 demonstrations), so they indicate the gap rather than measure it in a controlled comparison.

Table 1: Main result, Full-1000. Rows 2 and 3 are compared with row 1, rows 4 and 5 with the row above. Speed: playback speed relative to the demonstrations; Exec.: execution horizon; Succ.: successful episodes; settings: shared diffusion timestep and state dropout.

Step	Model	Speed, Exec.	Q	Succ.
1 Baseline	RGB only	1×, 16	0.246	77
2 + faster execution	same checkpoint as 1	2×, 16	0.279	124
3 + multi-stream	CYBERWAM, no settings	1×, 16	0.331	144
4 + faster execution	same checkpoint as 3	2×, 32	0.456	258
5 + training settings	CYBERWAM, with settings	2×, 32	**0.606**	**412**

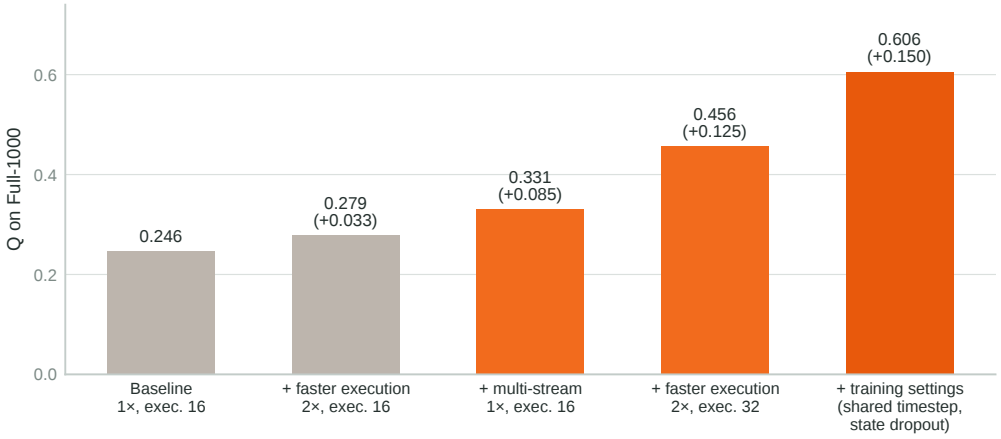


Figure 6: Main result on Full-1000 (Table 1). Grey: baseline; orange: multi-stream model. The gain in brackets is that of the change named under each bar.

4.3 Generalization beyond the training layout

In *turning on radio* (task 0), the radio stands on the table, at a height of 0.53 m, in all 300 training instances and in all 20 public instances of the task. We move the radio in public instance 301 from the table to the floor, about 1 m from the robot's start position, and keep the rest of the instance unchanged. The model has never seen the radio on the floor in this task. It still walks to the radio, picks it up from the floor and completes the task (Figure 7).

The policy therefore does not replay a trajectory memorized for this task: reaching down to the floor never occurs in the radio demonstrations. Other tasks do contain floor-level grasps, for example the teddy bears in *collecting children's toys* (task 21) and the shoes in *putting shoes on rack* (task 22). The model locates the radio from what it sees and combines the goal of one task with a motion learned in others. We take this as a sign that a WAM trained on all 100 tasks, on top of a backbone pretrained on large-scale video, learns a visual and motor representation shared across tasks rather than one behaviour per task, and can therefore act in layouts outside the distribution of a task's own demonstrations.

5 Conclusion

We presented our solution to the BEHAVIOR-1K 2026 challenge, a world action model that denoises future RGB video, DINO features, point maps and a chunk of robot actions with one video diffusion backbone and a separate action expert. Our experiments show that the score improves when training, model and deployment are optimized together. On the training side, we introduced state dropout and a shared diffusion timestep across streams. On the model side,

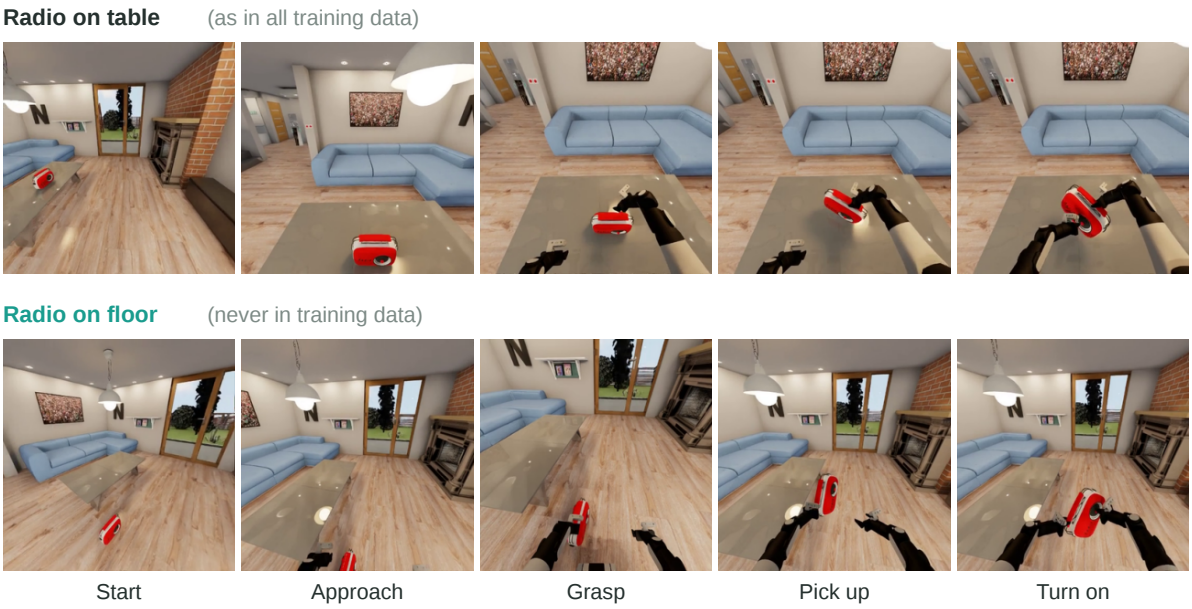


Figure 7: *Turning on radio*, public instance 301, head camera. *Top:* the original instance, with the radio on the table as in all training data. *Bottom:* the same instance with the radio moved to the floor, a layout that never occurs in the training data. The final model succeeds in both; the last column shows the step at which the radio is switched on.

we added a semantic stream (DINO features) and a depth stream (point maps) to the predicted future. On the deployment side, we studied different playback speeds.

We also ran post-training experiments, with SFT and with RFT. The post-trained models outperformed the pretrained model on only a few tasks. A better training recipe might make a single pretrained model outperform the post-trained models on all tasks, which we leave open.

Several directions remain open. Our generalization test (Section 4.3) moved one object into a layout that never occurs in its task's demonstrations, and the model still completed the task. Whether this holds more broadly is not yet known: a systematic study of out-of-distribution layouts, varying object positions, scenes and object instances across many tasks, would show how far such transfer reaches and how much of it comes from video pretraining and how much from the multi-task robot data. Our model also chooses actions from the current observation only, and we plan to add history information to it. Finally, limited by time and resources, we did not try the promising RSI direction, which we will explore next.

Acknowledgements

We thank the organizers of the BEHAVIOR-1K 2026 challenge for the benchmark, the demonstrations and the evaluation infrastructure, and the authors of Fast-WAM, Flex-π, Wan2.2 and DINOv3 for releasing their code and models.

References

[1] Chengshu Li, Ruohan Zhang, Josiah Wong, et al. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024. URL `https://arxiv.org/abs/2403.09227`.

[2] Ilia Larchenko, Gleb Zarin, and Akash Karnatak. Task adaptation of vision-language-action

model: 1st place solution for the 2025 behavior challenge. *arXiv preprint arXiv:2512.06951*, 2025. URL https://arxiv.org/abs/2512.06951.

[3] Junjie Bai, Yu-Wei Chao, Qizhi Chen, Jinwei Gu, Moo Jin Kim, Zhaoshuo Li, Xuan Li, Tsung-Yi Lin, Ming-Yu Liu, Nic Ma, Kaichun Mo, Delin Qu, Shangkun Sun, Hongchi Xia, Fangyin Wei, and Xiaohui Zeng. Openpi comet: Competition solution for 2025 behavior challenge. *arXiv preprint arXiv:2512.10071*, 2025. URL https://arxiv.org/abs/2512.10071.

[4] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-wam: Do world action models need test-time future imagination? *arXiv preprint arXiv:2603.16666*, 2026. URL https://arxiv.org/abs/2603.16666.

[5] Team Wan, Ang Wang, Baole Ai, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025. URL https://arxiv.org/abs/2503.20314.

[6] Ge Yan, Jinghao Liu, Yuzhi Fan, Lei Cai, Minwen Liao, Jesse Zhang, and Dieter Fox. Flex-π: A multi-stream world-action model with compute flexibility. *arXiv preprint arXiv:2608.10860*, 2026. URL https://arxiv.org/abs/2608.10860.

[7] Oriane Siméoni, Huy V. Vo, Maximilian Seitzer, et al. Dinov3. *arXiv preprint arXiv:2508.10104*, 2025. URL https://arxiv.org/abs/2508.10104.

[8] Yilun Du, Mengjiao Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Joshua B. Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. *arXiv preprint arXiv:2302.00111*, 2023. URL https://arxiv.org/abs/2302.00111.

[9] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. *arXiv preprint arXiv:2412.14803*, 2024. URL https://arxiv.org/abs/2412.14803.

[10] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. *arXiv preprint arXiv:2503.00200*, 2025. URL https://arxiv.org/abs/2503.00200.

[11] Yue Liao, Pengfei Zhou, Siyuan Huang, Donglin Yang, Shengcong Chen, Yuxin Jiang, Yue Hu, Jingbin Cai, Si Liu, Jianlan Luo, Liliang Chen, Shuicheng Yan, Maoqing Yao, and Guanghui Ren. Genie envisioner: A unified world foundation platform for robotic manipulation. *arXiv preprint arXiv:2508.05635*, 2025. URL https://arxiv.org/abs/2508.05635.

[12] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026. URL https://arxiv.org/abs/2601.16163.

[13] Hongzhe Bi, Hengkai Tan, Shenghao Xie, Zeyuan Wang, Shuhe Huang, Haitian Liu, Ruowen Zhao, Yao Feng, Chendong Xiang, Yinze Rong, Hongyan Zhao, Hanyu Liu, Zhizhong Su, Lei Ma, Hang Su, and Jun Zhu. Motus: A unified latent action world model. *arXiv preprint arXiv:2512.13030*, 2025. URL https://arxiv.org/abs/2512.13030.

[14] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, Nan Xue, Xing Zhu, Yujun Shen, and Yinghao Xu. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026. URL https://arxiv.org/abs/2601.21998.

[15] Javier Alejandro Lopetegui Gonzalez, Paul Pacaud, and Cordelia Schmid. Spatially aware world action model via geometric latent diffusion. *arXiv preprint arXiv:2609.02531*, 2026. URL https://arxiv.org/abs/2609.02531.

[16] Mutian Tong, Han Jiang, Qiao Feng, Lingjie Liu, and Jiatao Gu. Pointaction: 3d points as universal action representations for robot control. *arXiv preprint arXiv:2606.03943*, 2026. URL https://arxiv.org/abs/2606.03943.

[17] Haoyu Zhao, Xingyue Zhao, Siteng Huang, Xin Li, Deli Zhao, and Zhongyu Li. Rynnworld-4d: 4d embodied world models for robotic manipulation. *arXiv preprint arXiv:2607.06559*, 2026. URL https://arxiv.org/abs/2607.06559.

[18] Chongyang Xu, Haipeng Li, Shen Cheng, Jingyu Hu, Haoqiang Fan, Ziliang Feng, and Shuaicheng Liu. Action-geometry prediction with 3d geometric prior for bimanual manipulation. *arXiv preprint arXiv:2602.23814*, 2026. URL https://arxiv.org/abs/2602.23814.

[19] Tingting Du, Ziyao Wang, Guoheng Sun, and Ang Li. Xgenact: Geometry-enhanced world action models through cross-task generation. *arXiv preprint arXiv:2610.03516*, 2026. URL https://arxiv.org/abs/2610.03516.

[20] Chunghyun Park, Beomjun Kim, Seungcheol Park, Heeseung Kwon, Yashu Shukla, Seunghoon Sim, Jinwoo Shin, and Minsu Cho. Pointwam: 3d world action modeling for dexterous robotic manipulation. *arXiv preprint arXiv:2610.02840*, 2026. URL https://arxiv.org/abs/2610.02840.

[21] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022. URL https://arxiv.org/abs/2210.02747.

[22] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. URL https://arxiv.org/abs/2410.24164.

[23] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. URL https://arxiv.org/abs/2504.16054.

[24] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023. URL https://arxiv.org/abs/2303.04137.

[25] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023. URL https://arxiv.org/abs/2304.13705.

[26] Weixin Liang, Lili Yu, Liang Luo, Srinivasan Iyer, Ning Dong, Chunting Zhou, Gargi Ghosh, Mike Lewis, Wen tau Yih, Luke Zettlemoyer, and Xi Victoria Lin. Mixture-of-transformers: A sparse and scalable architecture for multi-modal foundation models. *arXiv preprint arXiv:2411.04996*, 2024. URL https://arxiv.org/abs/2411.04996.

A Implementation Details

A.1 Training data

We train on the BEHAVIOR-1K 2026 challenge demonstrations: 20 000 teleoperated episodes over the 100 tasks, recorded at 30 Hz (210.9M frames, about 1 950 hours). Each frame provides RGB and linear depth from the three cameras, the 61-D proprioception vector of the simulator and the 23-D action. The final model uses all episodes for training; no episode is held out. A training sample starts at an anchor frame t; anchors are placed every 6 frames of each episode, which gives about 35.2M samples per epoch. Around the anchor we read 32 actions at frames $t, t{+}2, \ldots, t{+}62$ (15 Hz), the state at frame t, and 9 video frames at $t, t{+}8, \ldots, t{+}64$ (3.75 Hz), which the VAE compresses into one anchor latent and two future latents. Indices past the end of an episode are clamped to its last frame and marked as padding.

A.2 Images and point maps

The head view is resized to 256×256 and each wrist view to 128×128 before tiling. Depth is decoded from the official 12-bit log quantization to metric depth in $[0.01, 10]$ m and unprojected with each camera's intrinsics into camera-frame XYZ at native resolution. Invalid pixels are set to zero. Coordinates are clipped to $[-1.5, 1.5] \times [-1.5, 1.5] \times [0, 3]$ m, scaled to $[-1, 1]$, resized with nearest-neighbour sampling and tiled in the same layout as the RGB views. DINO features are computed online from the tiled RGB image by the frozen encoder.

A.3 State, action and text

The 23-D state is taken from the 61-D vector: base velocity, torso and arm joint positions, and for each gripper the sum of its two finger positions. Continuous action dimensions and the state are z-scored per dimension with global statistics over the training set; actions are not clipped, and normalized states are clamped to ± 5. Gripper actions stay in their binary ± 1 form, and one torso dimension that is constant zero in the data is fixed to zero. The prompt is the task name inserted into a fixed template ("A video recorded from a robot's point of view executing the following instruction: ..."); its T5 embedding is precomputed once per task.

A.4 Optimization

We optimize with AdamW ($\beta = (0.9, 0.95)$, weight decay 0.01), a cosine schedule from 2×10^{-4} to 10^{-6} with a warm-up of 1288 steps (0.3 epoch), gradient clipping at 1.0 and bf16 precision, using DeepSpeed ZeRO-1. The final model is trained for 115 884 steps.

A.5 Inference and execution

At test time the model denoises with 20 sampling steps and returns 32 predicted actions. The policy executes all of them (execution horizon 32) and then re-plans. We execute them with `speed_2x`: each 15 Hz action is applied as one 30 Hz control step, so the 32 predicted actions form 32 control steps.

Post-processing. The predicted chunk is first de-normalized with the training statistics, and three rules turn it into physical commands. The one dimension that is constant zero in the data is set to zero. Each gripper command is binarized by its sign, with a threshold of 0, to ± 1. The base command (two linear velocities and the yaw rate) is clipped to $[-1, 1]$ and mapped to physical velocities with scales of 0.75, 0.75 and 1.0. Under `speed_2x`, each action is first repeated twice to the 30 Hz control rate. The base velocities are then integrated into an SE(2) path and re-sampled onto half as many control steps, so that the robot follows the same path in

half the time, and are divided by the scales and clipped to $[-1, 1]$ again. The other continuous dimensions (torso and arm targets) are linearly interpolated at the compressed time steps, and the gripper commands are held.

Depth at test time. The point-map stream needs depth at every step, so the observation must contain the depth images of the head camera (720×720) and of both wrist cameras (480×480) in addition to the RGB images. Non-finite depth values, which the simulator returns for rays that hit nothing, are replaced by the maximum depth, and all depths are clamped to $[0.01, 10]$ m, the range of the official depth encoding that the training data went through. The depth is then projected exactly as in training (Section A.2).

B Hyper-parameters

Table 2 lists the configuration of the final model. The corresponding training configuration is included in the code release.

Table 2: Hyper-parameters of the final model.

Group	Setting	Value
Video expert	Initialization	Wan2.2-TI2V-5B DiT
	Layers / width / FFN width	30 / 3072 / 14 336
	Attention heads × head dim.	24×128
	Patch size ($t \times h \times w$)	$1 \times 2 \times 2$, 48-channel latents
Action expert	Initialization	Wan2.2 DiT, interpolated to width 1024
	Layers / width / FFN width	30 / 1024 / 4096
	Attention heads × head dim.	24×128
Mixed attention	Stream-local layers	bottom 25% and top 25%
	Cross-stream layers	middle 50%
Inputs	Tiled camera image	384×256 (head 256^2, wrists 128^2)
	Video frames per sample	9 (1 anchor + 2 future latents)
	Action chunk	32 steps at 15 Hz (2.1 s), 23-D
	State	23-D, current frame only
	Text	T5 embedding of the task prompt, 128 tokens
Flow matching	Training timesteps	1000
	Schedule shift	5 for all streams
	Timestep across streams	shared
	DINO stream target	clean features (x-prediction)
	Loss weights	1 for every stream
Regularization	State dropout	10% of samples
Optimization	Optimizer	AdamW, $\beta = (0.9, 0.95)$, w.d. 0.01
	Learning rate	2×10^{-4} cosine to 10^{-6}, 1288 warm-up
	Gradient clipping	1.0
	Precision / parallelism	bf16 / DeepSpeed ZeRO-1
	Batch size per GPU	32
	Training steps	115 884
Execution	Playback speed	2× (one control step per action)
	Execution horizon	32 actions

C Per-task Results

Table 3 lists the score of our challenge submission on each task, averaged over its 10 public test instances, together with the number of successful episodes. Averaged over the 100 tasks, the submission reaches 0.606 Q with 412 successes out of 1000 episodes.

D Additional Visualizations

Figure 8 shows keyframes of eleven long successful rollouts of the final model on the public test instances. Each task takes between 7 and 14 minutes of simulated time and several thousand control steps, and requires the robot to move between rooms and to manipulate several objects in turn.

Table 3: Per-task results on the 1000 public test rollouts. #: task index; Q: mean score over the 10 instances; S: successful episodes out of 10.

#	Task	Q	S
0	turning on radio	0.70	7
1	picking up trash	0.87	6
2	putting away Halloween decorations	0.61	5
3	cleaning up plates and food	0.29	1
4	can meat	0.23	0
5	setting mousetraps	0.72	4
6	hiding Easter eggs	0.61	5
7	picking up toys	0.63	4
8	rearranging kitchen furniture	0.82	7
9	putting up Christmas decorations inside	0.56	0
10	set up a coffee station in your kitchen	0.53	2
11	putting dishes away after cleaning	0.64	4
12	preparing lunch box	0.68	2
13	loading the car	0.80	7
14	carrying in groceries	0.68	5
15	bringing in wood	0.23	1
16	moving boxes to storage	1.00	10
17	bringing water	1.00	10
18	tidying bedroom	0.90	7
19	outfit a basic toolbox	0.51	3
20	sorting vegetables	0.57	0
21	collecting children's toys	0.89	5
22	putting shoes on rack	0.91	5
23	boxing books up for storage	0.45	1
24	storing food	0.79	1
25	clearing food from table into fridge	0.88	8
26	assembling gift baskets	0.68	0
27	sorting household items	0.31	0
28	getting organized for work	0.16	1
29	clean up your desk	0.53	0
30	setting the fire	0.47	0
31	clean boxing gloves	0.95	9
32	wash a baseball cap	0.95	9
33	wash dog toys	0.53	3
34	hanging pictures	0.90	9
35	attach a camera to a tripod	0.40	4
36	clean a patio	0.10	1
37	clean a trumpet	0.00	0
38	spraying for bugs	0.45	2
39	spraying fruit trees	0.40	4
40	make microwave popcorn	0.70	7
41	cook cabbage	0.45	2
42	chop an onion	0.70	4
43	slicing vegetables	0.74	5
44	chopping wood	0.40	2
45	cook hot dogs	1.00	10
46	cook bacon	0.41	1
47	freeze pies	0.43	3
48	canning food	0.16	0
49	make pizza	0.00	0
50	freeze fruit	0.16	1
51	cook a brisket	0.70	7
52	sorting bottles cans and paper	0.28	1
53	tidying living room	0.68	6
54	putting away toys	0.91	5
55	re-shelving library books	0.73	5
56	make rose centerpieces	0.78	5
57	sweeping garage	0.20	1
58	stacking wood	0.97	8
59	organizing art supplies	0.60	3
60	scrubbing bathroom floor	0.70	7
61	bringing paper to recycling	0.63	6
62	halve an egg	0.48	3
63	installing smoke detectors	0.20	2
64	setting the table	0.47	3
65	unloading the car	0.95	9
66	turning out all lights before sleep	0.82	4
67	boxing food after dinner	0.57	2
68	cleaning up branches and twigs	0.42	3
69	vacuuming floors	0.10	1
70	thawing frozen food	0.26	0
71	clean your rusty garden tools	0.30	1
72	cook a frozen pie	0.70	7
73	organizing school stuff	0.78	3
74	carrying out garden furniture	0.75	6
75	put together a basic pruning kit	0.90	8
76	dispose of glass	1.00	10
77	installing a modem	0.78	6
78	make cabinet doors	0.20	2
79	polishing shoes	0.40	0
80	clean up broken glass	0.90	8
81	packing meal for delivery	0.65	1
82	store batteries	1.00	10
83	store honey	1.00	10
84	tidying bathroom	0.78	4
85	putting dirty dishes in sink	0.97	9
86	make gift bags for baby showers	0.52	2
87	collecting aluminum cans	0.78	3
88	rearrange your room	0.73	5
89	installing a fax machine	0.70	7
90	composting waste	0.85	7
91	store produce	0.85	7
92	installing a scanner	0.05	0
93	clean a keyboard	0.40	4
94	dispose of batteries	0.90	8
95	cook brussels sprouts	0.26	1
96	cook broccolini	0.48	3
97	setup a bar for a cocktail party	0.81	2
98	laying tile floors	0.96	9
99	sorting books on shelf	0.23	1

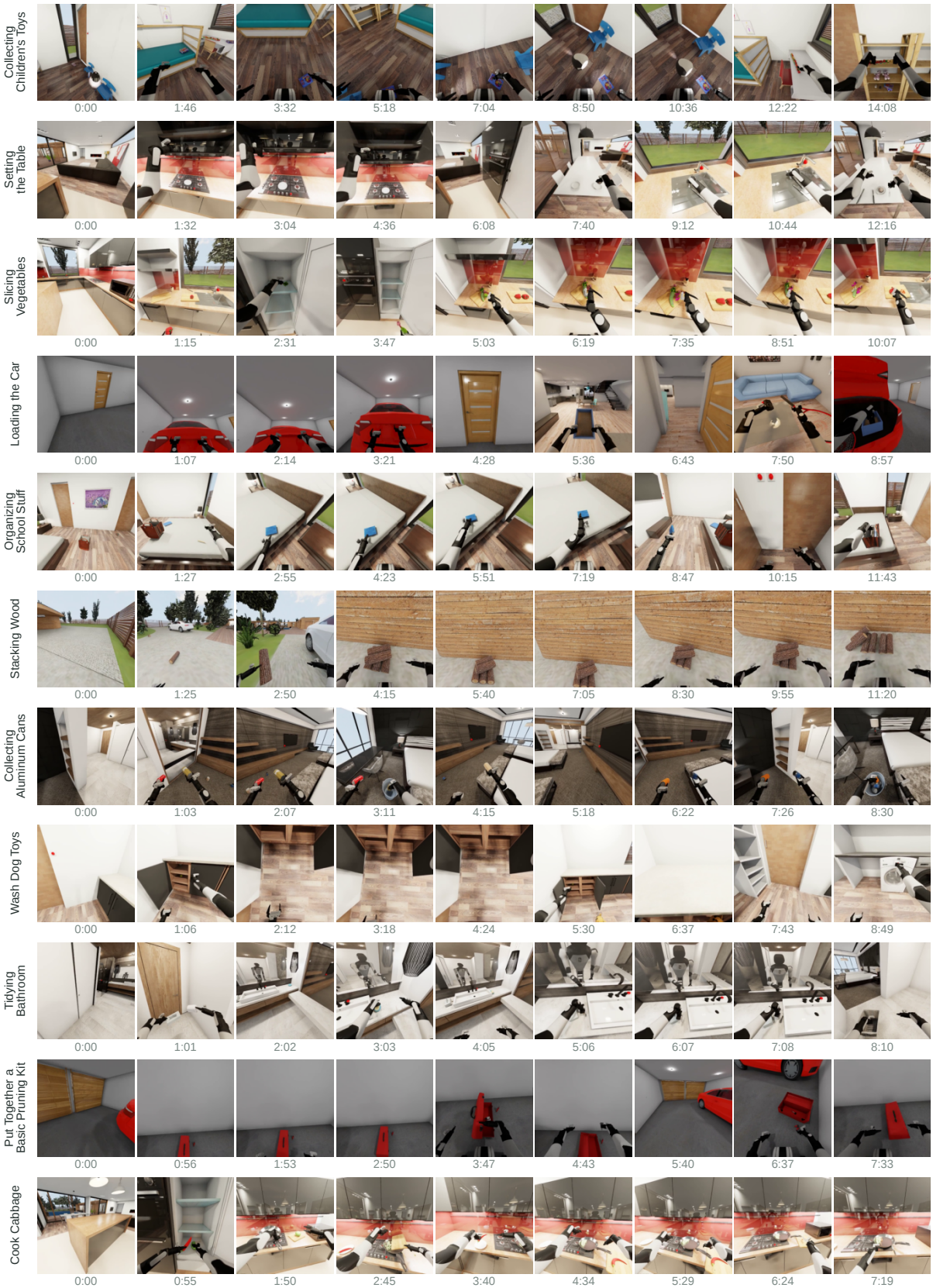


Figure 8: Keyframes of successful rollouts of the final model, head camera, nine evenly spaced frames per episode. The time below each frame is simulated time since the start of the episode.